

Algoritmo de Dijkstra

Resumo

O algoritmo de Dijkstra é um dos algoritmos mais utilizados para resolver o problema de caminhos mínimos em grafos. Seu objetivo é encontrar a menor distância entre um vértice de origem e todos os outros vértices do grafo, desde que não existam arestas com pesos negativos.

Implementação em C

Nesta implementação, o grafo é representado por uma matriz de adjacência, onde cada posição `matriz[i][j]` armazena o peso da aresta entre os nós `i` e `j`. Caso não exista conexão entre dois nós, o valor armazenado é 0.

A função principal do algoritmo recebe três parâmetros:

- `matriz`: matriz de adjacência do grafo;
- `num_nos`: quantidade total de nós (vértices);
- `origem`: nó inicial a partir do qual as distâncias serão calculadas.

1. Inicialização

Inicialmente, são criados dois vetores auxiliares:

```
int *dist;  
bool *visitado;
```

- `dist`: armazena a menor distância conhecida entre a origem e cada vértice do grafo.
- `visitado`: indica quais vértices já tiveram sua menor distância definitivamente determinada.

Essa etapa corresponde ao seguinte trecho de pseudocódigo:

```
para i ← 0 até n-1 faça  
    dist[i] ← INFINITO  
    visitado[i] ← falso  
fim-para
```

No código em C, todas as distâncias são inicializadas com `INT_MAX / 2`, representando o infinito. O uso da divisão por dois (`/ 2`) evita problemas de *overflow* durante as somas realizadas no relaxamento das arestas.

Em seguida, define-se a distância da origem para ela mesma, que é sempre zero:

```
dist[origem] = 0;
```

2. Laço Principal e Seleção do Vértice

Após a inicialização, o algoritmo entra em seu laço principal, que será executado `num_nos - 1` vezes.

Dentro desse laço, o primeiro passo é encontrar o vértice não visitado que possui a menor distância conhecida até o momento:

```
menor ← INFINITO
u ← -1

para i ← 0 até n-1 faça
    se visitado[i] = falso E dist[i] < menor então
        menor ← dist[i]
        u ← i
    fim-se
fim-para
```

O vértice escolhido (`u`) será aquele cuja menor distância já pode ser considerada definitiva. Caso nenhum vértice válido seja encontrado (ou seja, `u == -1`), o algoritmo é encerrado.

Depois disso, o nó selecionado é marcado como visitado:

```
visitado[u] = true;
```

Isso significa que o menor caminho até esse vértice já foi determinado e não será mais alterado.

3. Relaxamento das Arestas

A próxima etapa é o relaxamento das arestas, considerado o coração do algoritmo de Dijkstra.

O algoritmo verifica todos os vizinhos do vértice atual `u` e testa se existe um caminho menor passando por ele:

```
para cada vizinho v de u faça
    se dist[u] + peso(u,v) < dist[v] então
        dist[v] ← dist[u] + peso(u,v)
    fim-se
fim-para
```

O processo de relaxamento atualiza as menores distâncias conhecidas ao longo da execução do algoritmo. Ao final do processamento, o vetor `dist` conterá as menores distâncias da origem até todos os outros vértices alcançáveis do grafo.

4. Cálculo do Custo Total

Nesta implementação específica, em vez de retornar o vetor de distâncias, o algoritmo soma todas as menores distâncias encontradas:

```

custo ← 0

para i ← 0 até num_nos - 1 faça
    se dist[i] != INFINITO então
        custo ← custo + dist[i]
    fim-se
fim-para

```

Essa soma é utilizada como uma métrica de custo total do grafo a partir da origem.

5. Finalização

Por fim, a memória alocada dinamicamente é liberada com a função `free`, evitando vazamentos de memória (*memory leaks*):

```

free(dist);
free(visitado);

```

Complexidade

A implementação descrita possui complexidade $O(V^2)$, pois utiliza uma matriz de adjacência e uma busca linear para encontrar o próximo vértice de menor distância.

Para grafos grandes e esparsos, uma implementação otimizada utilizando listas de adjacência e uma fila de prioridade (*min-heap*) pode reduzir a complexidade para $O((V + E) \log V)$, sendo V o número de vértices e E o número de arestas.